Software Engineer, AI Applications — Take-Home Assignment

Thanks for taking the time to speak with us about the Software Engineer, AI Applications role at Hyperproof. This assignment reflects the work our AI team does every day: building agentic features on top of language models, and working in codebases where much of the code was written with AI tools.

You won't build something from scratch. You'll inherit a small AI-powered application that works, mostly, and make it something you'd be comfortable shipping to customers. We care much more about your judgment, the correctness of your changes, and how clearly you explain your decisions than about how much you add. A small set of changes that are correct, tested, and well explained will always score better than a large rewrite.

Time expectation: aim for **under 2 hours** of focused work. It's fine to leave TODOs or skip things, as long as you write down what you skipped and why.

Background

Northwind Labs, a fictional company, has an internal scheduling assistant. An employee types a request in plain English, such as "find 45 minutes with Priya and Sam next Tuesday afternoon." A language model reads the request and calls tools. The tools look up calendars and work out when everyone is free. The model then answers the employee.

The code has three layers:

- **A deterministic core** that models the working day as fixed-size time slots and searches for free windows.
- **Tools** that the model can call: list people, get someone's busy times, and find free slots for a group.
- **An agent loop** that sends the conversation to the model, runs the tools it asks for, and returns its final answer.

The application was written quickly by an AI coding agent. It runs, and its tests pass, but no one has reviewed it carefully. Treat it the way you would treat code a teammate handed to you before going on leave: you now own it.

Getting started

The starter code comes in two equivalent versions. Pick the one you're most comfortable with; they contain the same application and we evaluate them the same way.

Version	Requirements	Run the tests
Python	Python 3.10+, pytest	`python -m pytest`
TypeScript	Node.js 20+	`npm install`, then `npm test`

Each repository has a README covering the layout and how to run the command-line demo.

You do not need access to a language model. The starter includes `FakeModel`, which returns scripted responses. Because each scripted response is raw text, you can use it to simulate both well-formed replies and malformed or unexpected ones. All of our evaluation uses `FakeModel`.

If you'd like to try a real model, the starter includes an adapter for any OpenAI-compatible endpoint, including a free local server such as Ollama. This is entirely optional and earns no extra credit. We don't want access to paid AI services or powerful hardware to affect anyone's result.

What to do

There are three parts. The suggested times add up to about 1 hour 45 minutes.

Part A: Make the assistant trustworthy (about 60 minutes)

We'd like to put this assistant in front of real employees. Before we do, it must meet these requirements:

1. **Correct results.** Every time the assistant offers must actually work for every attendee, and it must not miss times that do work.
2. **No invented people.** The assistant must never propose a meeting that includes someone who isn't in the directory. If a request names someone it can't find, the user should hear about it.
3. **Graceful failure.** Language models sometimes return malformed output, call tools that don't exist, or pass bad

arguments. The assistant must handle these cases deliberately instead of crashing or quietly producing a wrong answer.
4. **It always finishes.** No request should be able to keep the assistant running indefinitely.

Fix any bugs you find along the way. Add tests that would have caught them, using `FakeModel` for anything involving the model. If you change an existing test, explain why in your notes.

Part B: Your own addition (about 30 minutes)

Choose **one** improvement that a real user of this assistant would want, implement it, and test it. We are deliberately not suggesting ideas: choosing what to build is part of the exercise. Keep it small enough to do well in the time.

Part C: NOTES.md (about 15 minutes)

Add a `NOTES.md` file at the root of the repository. Keep it short; bullet points are fine. Include:

- **Problems you found:** what each one was, how you found it, and what you changed.
- **Your addition:** what you built in Part B, who it helps, and why you chose it over other options.
- **How you used AI tools:** what you delegated, and at least one specific case where you rejected or corrected an AI suggestion, with your reason.
- **What you'd do next:** what you'd change or add with more time, and anything you noticed but deliberately left alone.

Using AI tools

Please use whatever AI coding tools you normally use. Our engineers use them every day, and we want to see how you work with them.

You are responsible for every line you submit, whoever or whatever wrote it. The live interview takes place in your own submitted code. Part of it is done without AI tools, and in that part we'll ask you to explain how the code works and to make changes to it. If you understand your submission well, that part will feel straightforward.

Out of scope

Please don't spend time on any of the following. None of it affects your evaluation:

- A user interface. The command-line demo is enough.
- Booking meetings, sending invitations, or writing to calendars.
- Time zones, recurring meetings, or calendar data beyond the sample files.
- Authentication, deployment, CI/CD, or containerization.
- Connecting to a real model, as explained in Getting started.
- Rewriting the application in another language or framework.

Deliverables

- **A link to a GitHub repository** containing your version of the starter. It can be public, or private and shared with the reviewer named in your invitation.
- **Your commit history.** Please commit as you go rather than in one final commit. Seeing the order in which you worked helps us understand your decisions.
- **NOTES.md**, as described in Part C.
- **Tests that pass** with the starter's single test command. If you changed how to run anything, update the README.

How we'll evaluate this

We'll look mainly at:

- **Correctness:** whether the assistant's results are right, including at the edges.
- **Robustness:** how deliberately you handle the ways a language model can misbehave.
- **Tests:** whether your tests check the behavior that matters and would catch real regressions.
- **Judgment:** what you chose to change, what you chose to leave alone, and why.
- **Your addition:** whether it solves a real user problem, and whether it's built and tested well.
- **Communication:** how clearly NOTES.md explains your reasoning.

If your submission moves forward, the next step is a one-hour live session. We'll spend about 45 minutes working in your

submitted code together, and leave the last 15 minutes for your questions about Hyperproof and the team.

Some things here are intentionally left open. Making a reasonable assumption, writing it down, and moving on is a normal part of this job. If you'd rather ask a clarifying question first, reply to your invitation email and we'll get back to you within one business day.

Good luck. We're looking forward to seeing what you build.